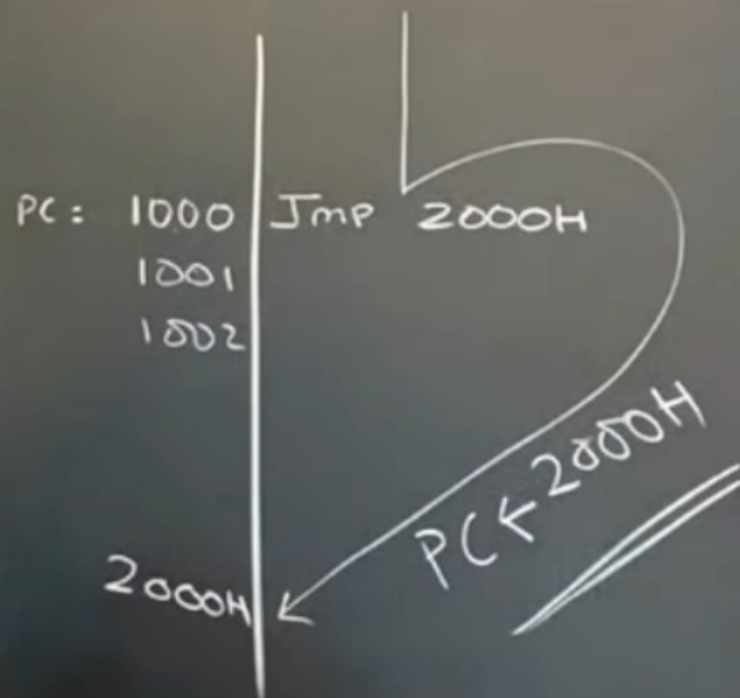


✓ 1) Jmp addr
Eg: `Jmp 2000H`



2) Conditional

`JC`
`JNC`

`JZ`
`JNZ`

`JPE`

`JPO`

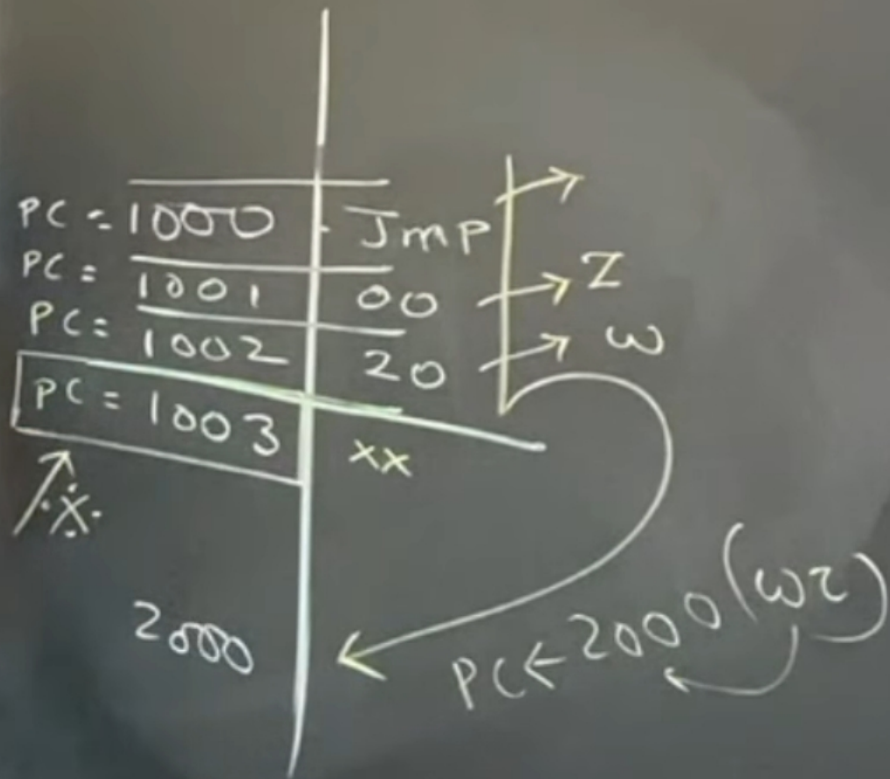
`JM`

`JP`



✓ 1) Jmp addr
Eg: Jmp 2000H

2) Con





✓
Eg: Jmp 2000H

Jmp 5678H

PC = 1234	Jmp →
PC = 1235	78 → Z
PC = 1236	56 → W
PC = 1237	xx

5678

PC ← 5678 (WZ)

JC
JNC

JZ
JNZ

JPE

JPO

JM

JP



JC 2000

2) Conditional Jmps

methods of

∞ loop

JC CF = 1

JNC CF = 0

JZ ZF = 1

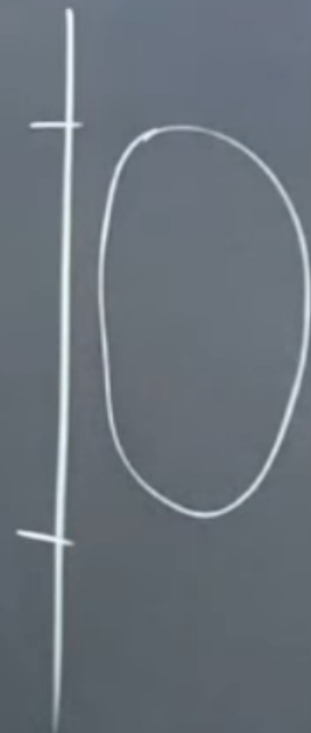
JNZ ZF = 0

JPE PF = 1

JPO PF = 0

JM SF = 1

JP SF = 0



✓ 1) Jmp addr

Ex: 20H

5678H

PC = 12

PC =

P



→ Z

2)

5678 (wz)

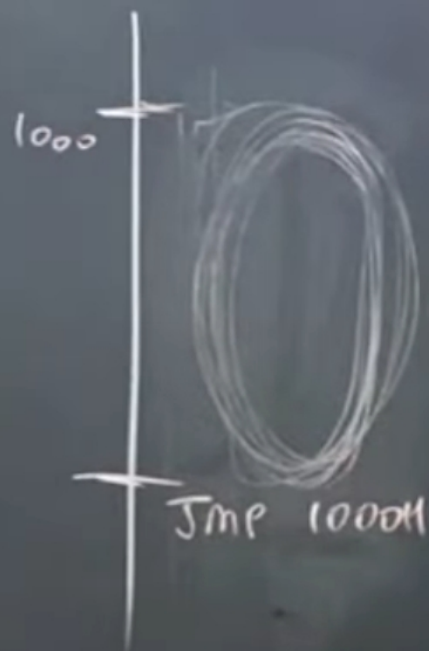
conditional Jmps

JC CF = 1
JNC CF = 0
JZ ZF = 1
JNZ ZF = 0
JPE PF = 1
JPO PF = 0
Jm SF = 1
JP SF = 0

methods of Looping

∞ loop

finite loop



PC: 1000 Call 2

1001

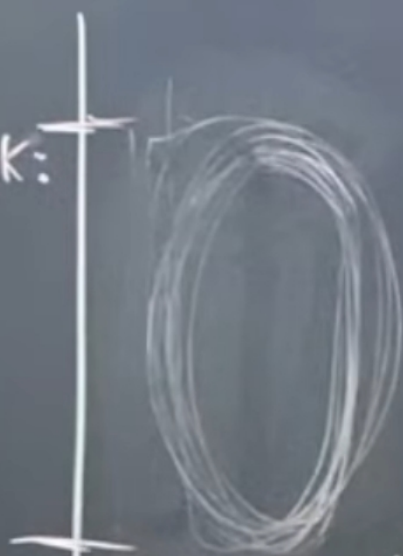
1002

(RA) 1003

CF = 1
CF = 0
ZF = 1
ZF = 0
PF = 1
PF = 0
SF = 1
SF = 0

∞ loop

Back:



Jmp Back

finite loop

PC: 1000

1001

1002

(RA) 1003

2) Conditional Jmps

JC C
JNC C
JZ Z
JNZ Z

∞ loop

Back:

Jmp Back

∞

finite loop

MVI C, 05H

Back:

'5'

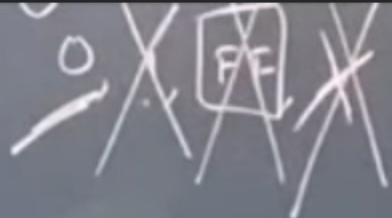
DCR C 4 3 2 1 0

JNZ Back

BONUS

Notes at the end of the video!

finite loop



Back: | mvi B, 08H

Back:

8



7 6 5 4 3 2 1 0

DCR B

JNZ Back

FF

PC: 10

10

10

(RA) 10

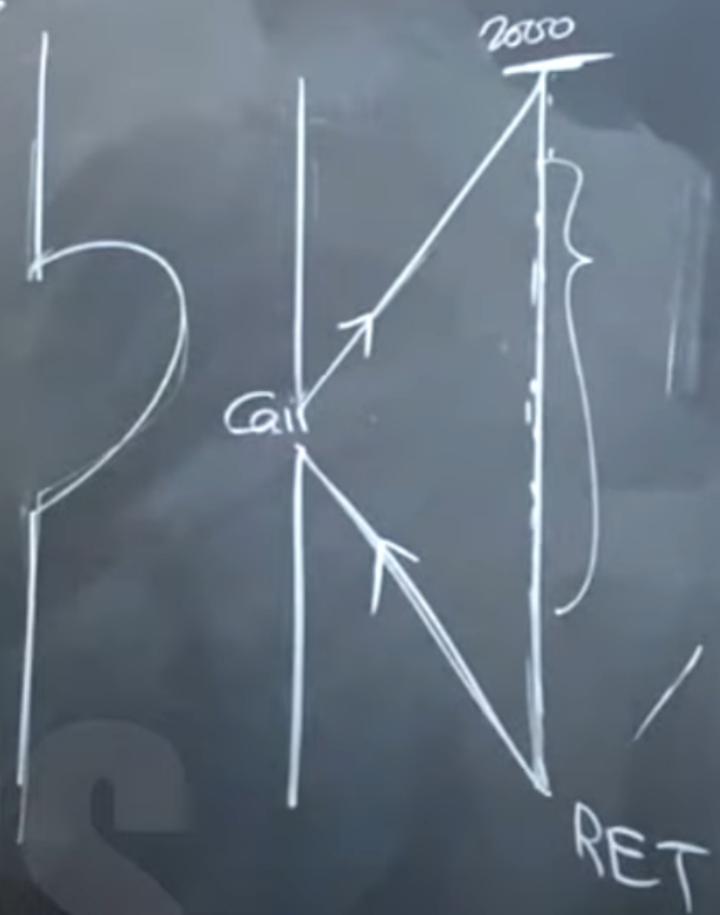
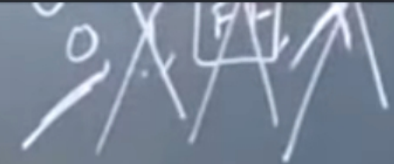
2000

Control Jumps

- CF = 1
- CF = 0
- ZF = 1
- ZF = 0
- PF = 1
- PF = 0
- SF = 1
- SF = 0

methods of Looping

infinite loop finite



PC = 1000

Call 2000

1001

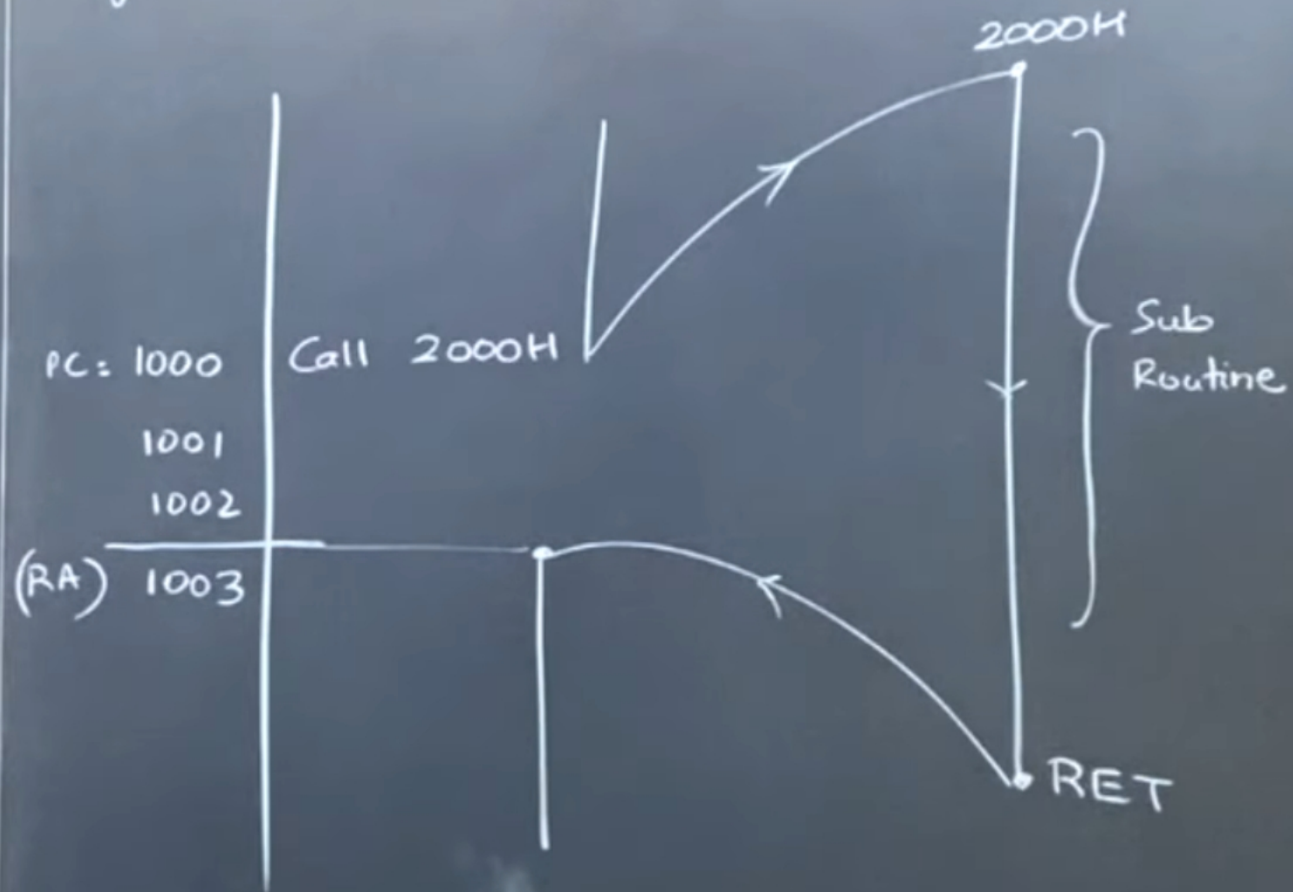
1002

(RA) 1003

BONUS

Notes at the end of the video!

✓ 0 X FF X



ds of Looping
finite

✓ 0 X ~~FF~~ X

Eg: Call 2000H

PC: 1000	Call
PC: 1001	00
PC: 1002	20
PC: (RA) 1003	xx

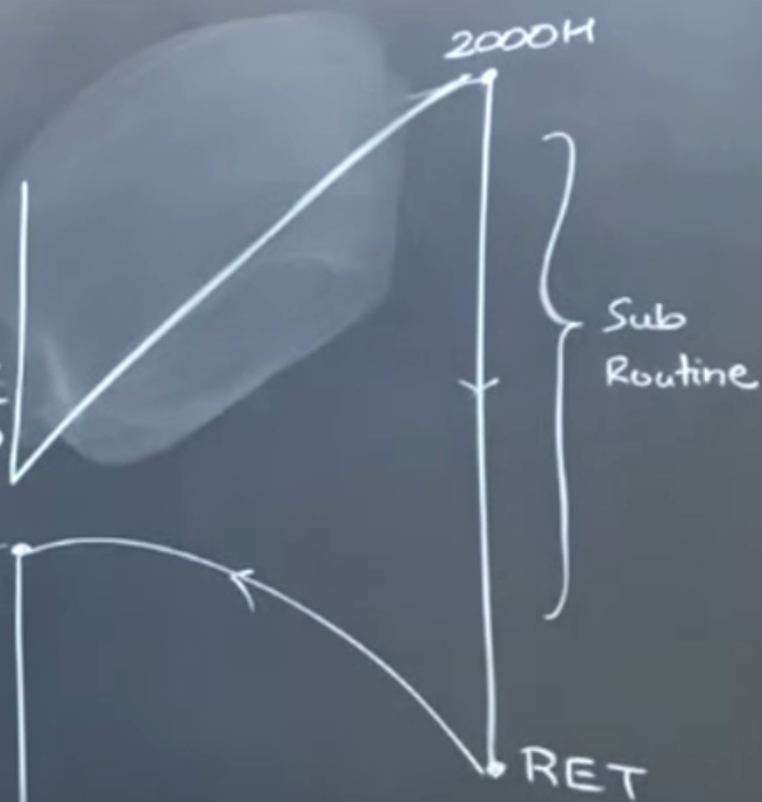
Call

00

20

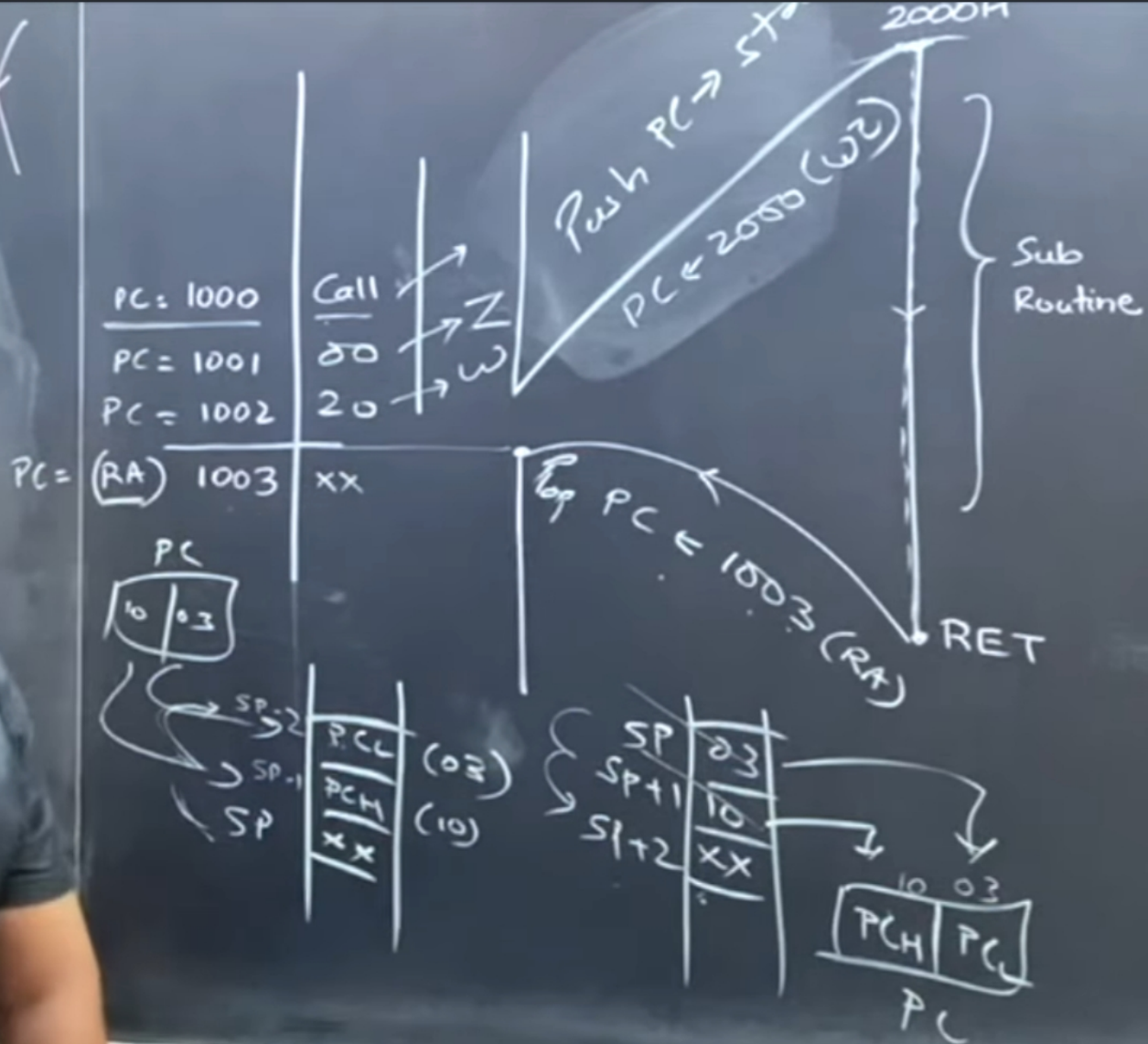
xx

→ Z
→ W

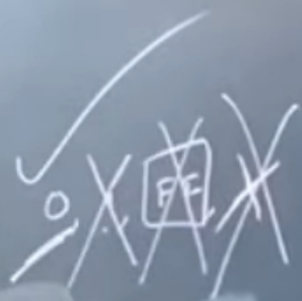


looping
finite

✓
~~0~~
~~X~~
~~FE~~
~~X~~



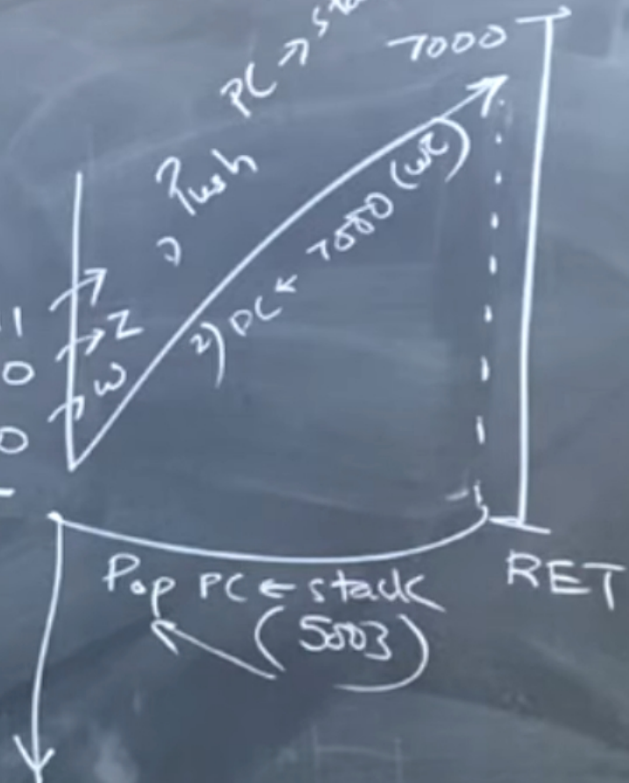
Methods of Looping
finite



3) Call address
Eg: Call 2000H

PC = 5000
PC = 5001
PC = 5002
PC = 5003
(RA)

Call
00
70

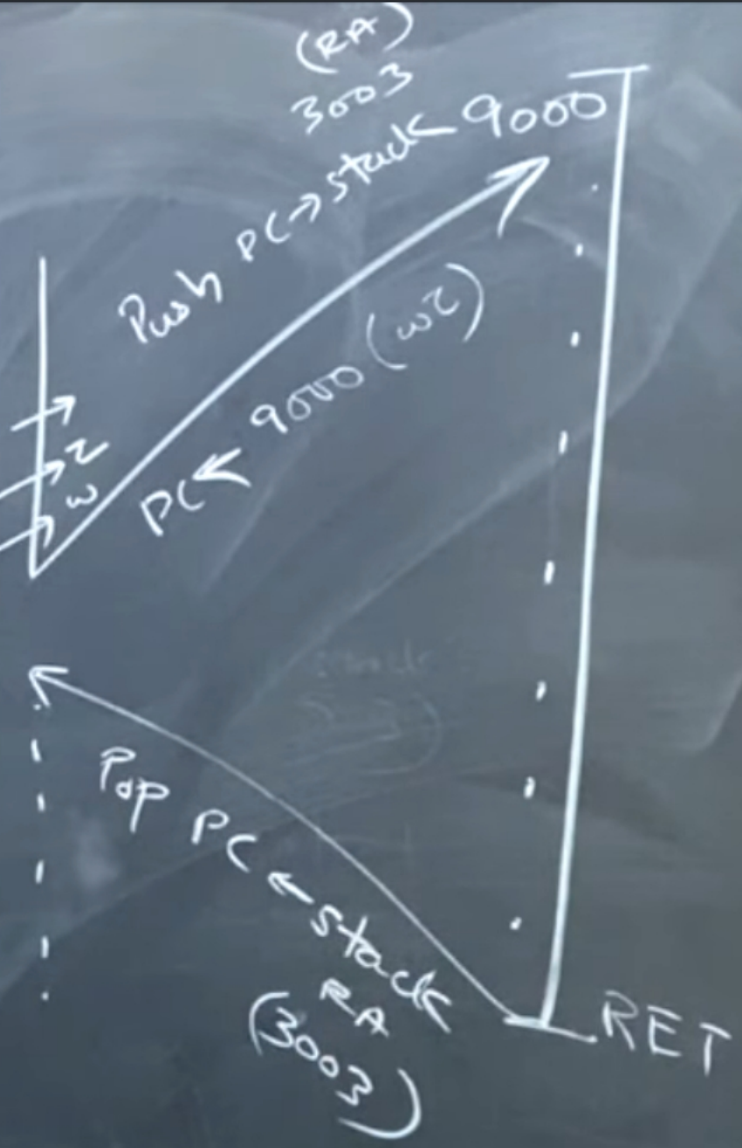


Looping
finite

✓
~~0~~
~~X~~
~~FE~~
~~X~~

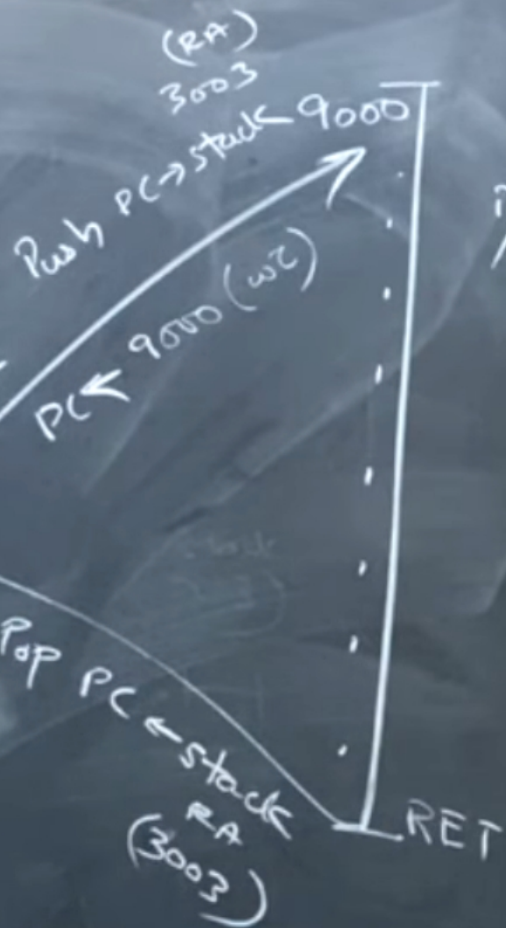
PC = 3000
PC = 3001
PC = 3002
PC = 3003

611
00
90
xx



PC = 3000
 PC = 3001
 PC = 3002
PC = 3003

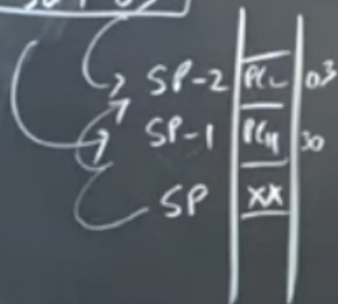
Call
 00
 90
 xx



Call 9000H

1) Push PC $\rightarrow$ stack
 PC

PC _H	PC _L
30	03



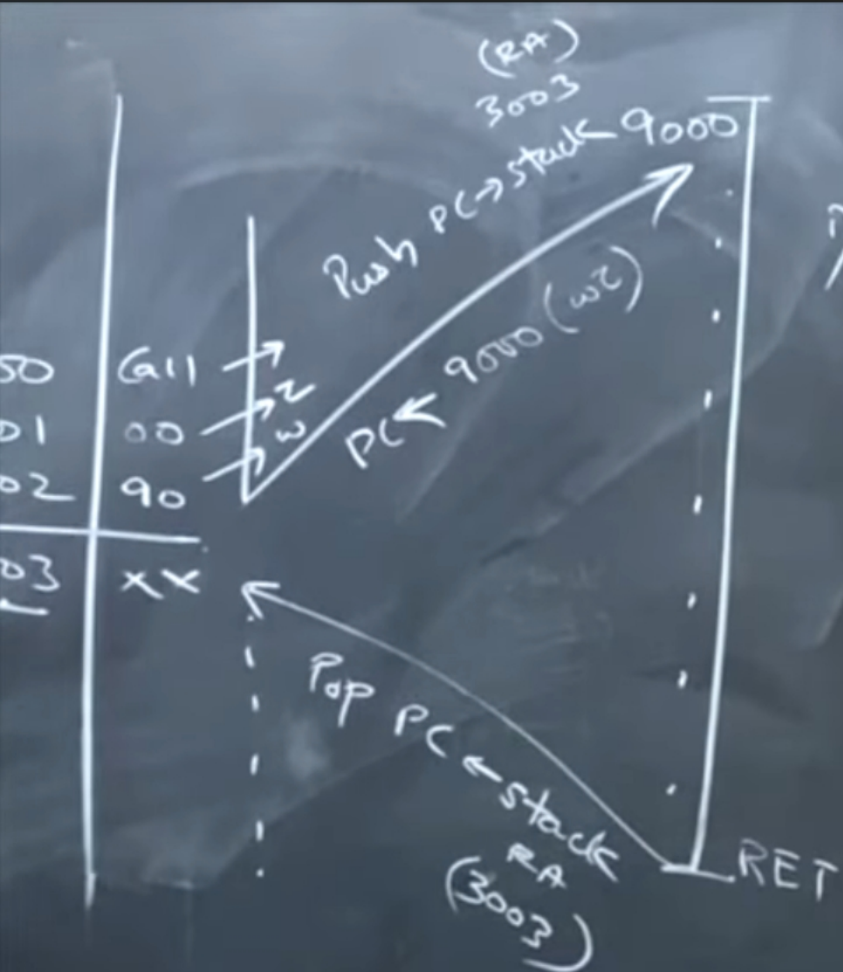
SP $\leftarrow$ SP-1

[SP] $\leftarrow$ PC_H (30)

SP $\leftarrow$ SP-1

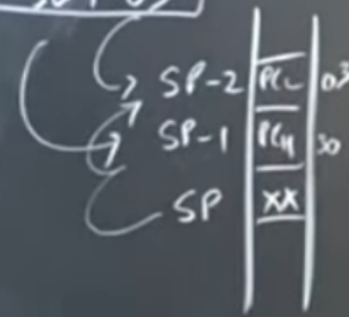
[SP] $\leftarrow$ PC_L (03)

2) PC $\leftarrow$ 9000 (wZ)



Call 9000H
 1) Push PC → stack
 PC

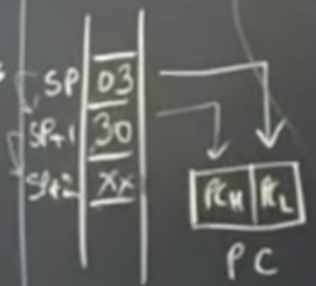
PC _H	PC _L
30	03



SP ← SP-1
 [SP] ← PC_H (30)
 SP ← SP-1
 [SP] ← PC_L (03)
 2) PC ← 9000 (wZ)

RET
 Pop PC ← stack

PC = 1000	RST
1001	



PC_L ← [SP] (03)
 SP ← SP+1
 PC_H ← [SP]
 SP ← SP+1

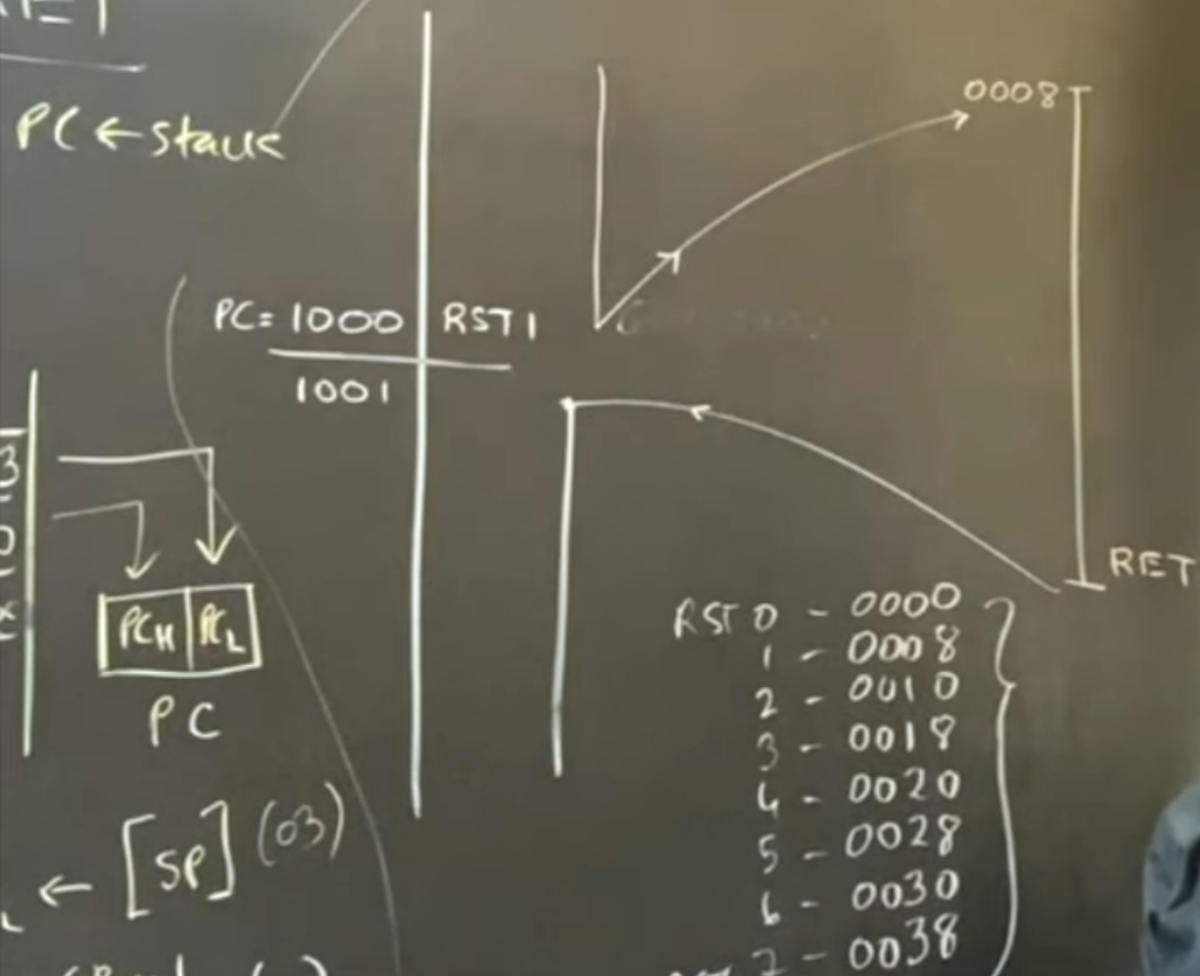


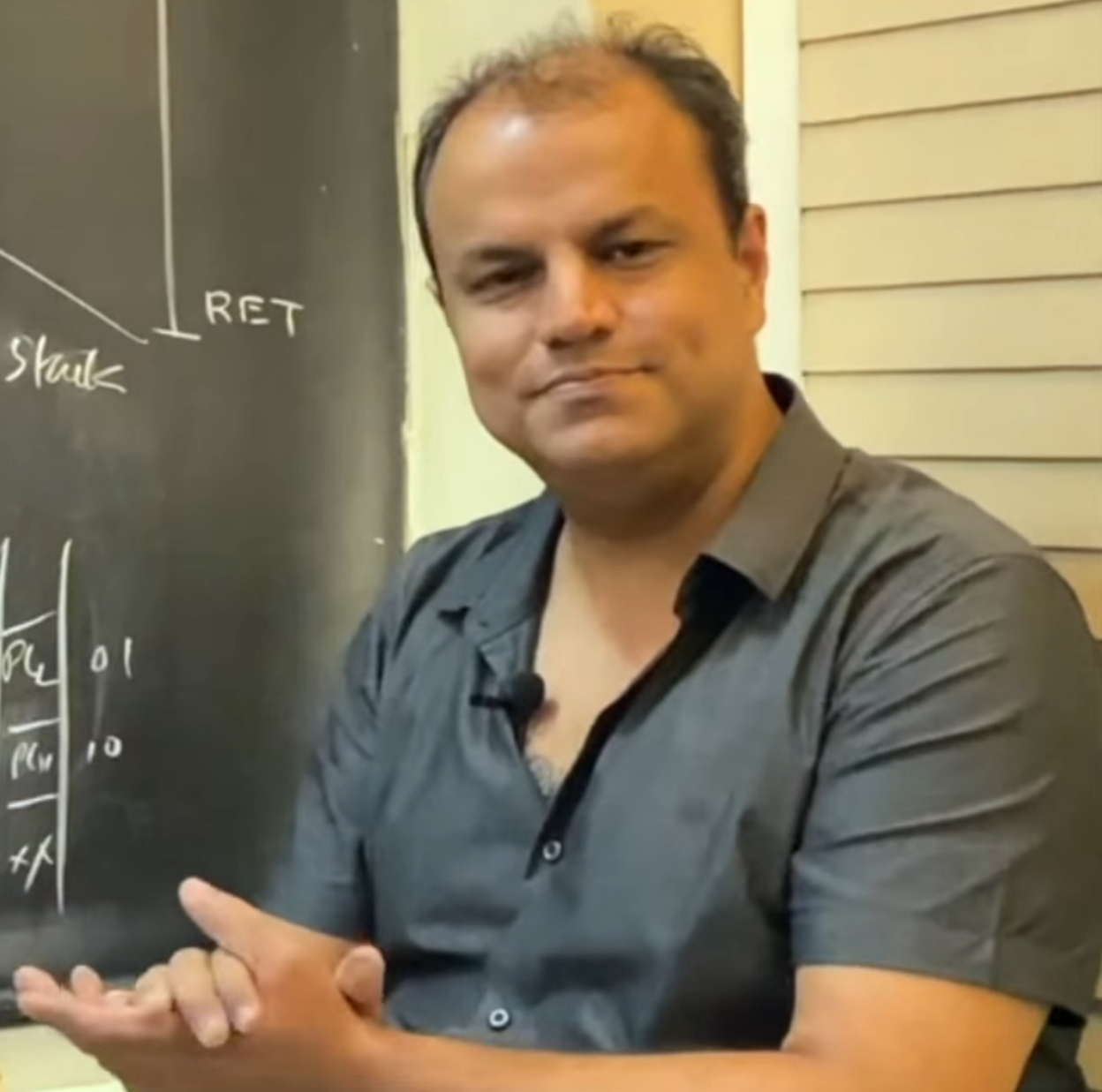
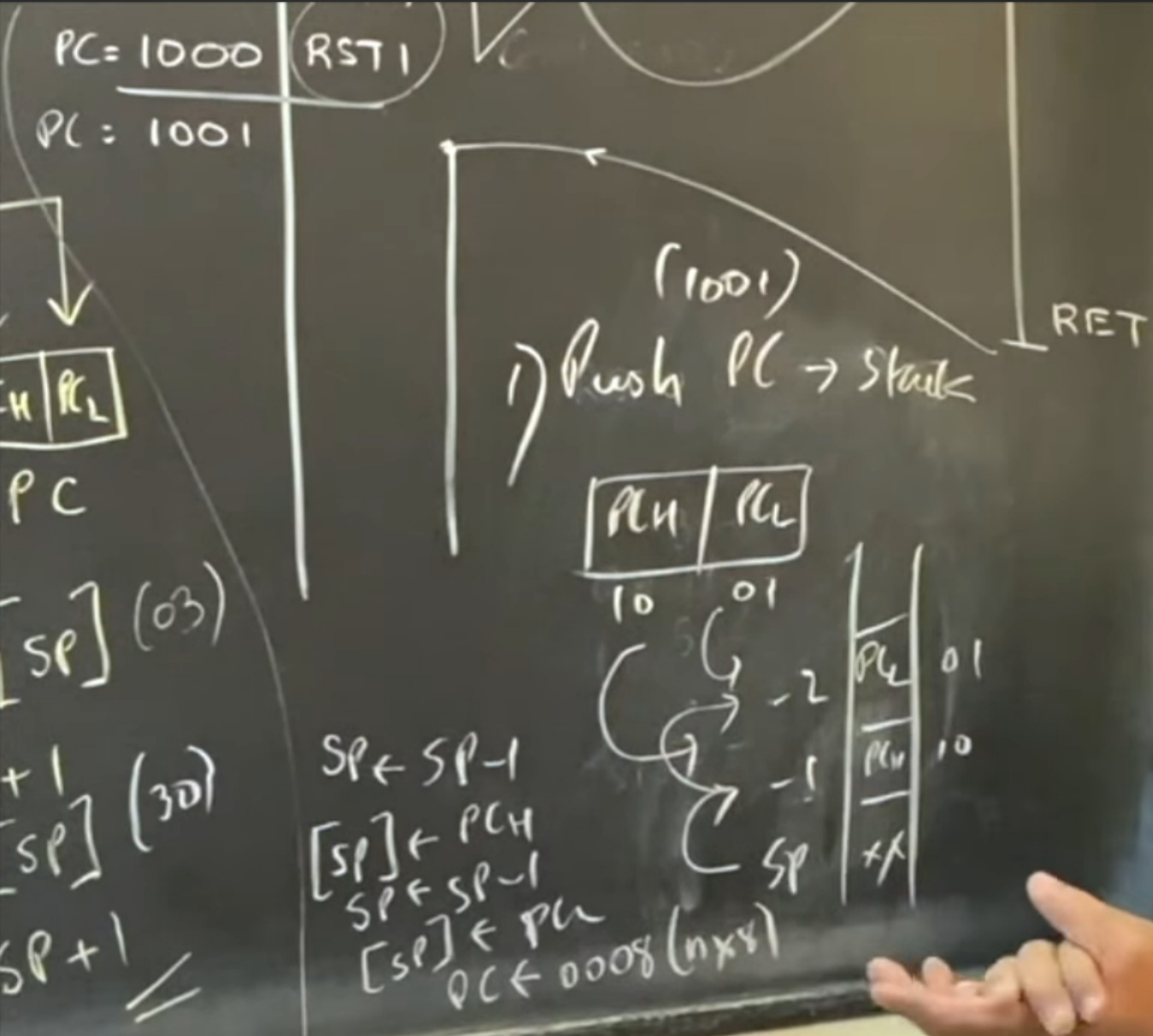
$RST_n \approx \text{Call}(n \times 8)$ { Software interrupts of 8085 }
 $(n = 0 \dots 7)$

$RST1 \approx \text{Call } 0008H$

RET

$PC \leftarrow \text{stack}$





BRANCH CONTROL GROUP

Special Note:

In the explanation of every instruction, I have mentioned its machine cycles and T-states. You will understand this part once you watch the two videos of timing diagrams

1) JMP 16-bit address (Unconditional Jump)

Loads PC with the 16-bit address specified in the instruction.

Eg: **JMP 2500** ; PC ← 2500

Addr. Mode	Flags Affected	Cycles	T-States
Immediate	None	3	10

Special Note:

Remember the point discussed in the video...

PC will contain address of the next instruction before it gets the branch address from WZ pair.

2) JCondition 16-bit address (Conditional Jump)

It is the same as UnConditional JUMP except that the action takes place ONLY if the condition is true.

Eg: **JZ 2500** ; PC ← 2500 if Z =1

Addr. Mode	Flags Affected	Cycles	T-States
Immediate	None	2/3	7/10

Conditions

Condition	Description	True if:
NZ	No Zero	Z=0
Z	Zero	Z=1
NC	No Carry	C=0
C	Carry	C=1
PO	Parity Odd	P=0
PE	Parity Even	P=1
P	Plus	S=0
M	Minus	S=1

3) CALL 16-bit address

(Unconditional Call)

Loads PC with the 16-bit address specified in the instruction.
Before doing so, it also Pushes the Current PC into the Stack.

Eg: **JMP 2500** ; $SP \leftarrow SP - 1$
 $[SP] \leftarrow PC_H$
 $SP \leftarrow SP - 1$
 $[SP] \leftarrow PC_L$
 $PC \leftarrow 2500$

#Please refer Bharat Sir's video for clear understanding of this instruction ...

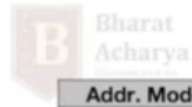
Addr. Mode	Flags Affected	Cycles	T-States
Immediate	None	5	18

4) CCondition 16-bit address

(Conditional Call)

It is the same as UnConditional Call except that the action takes place ONLY if the condition is true.

Eg: **CNZ 2500** ; IF $Z = 0$ then
 $SP \leftarrow SP - 1$
 $[SP] \leftarrow PC_H$
 $SP \leftarrow SP - 1$
 $[SP] \leftarrow PC_L$
 $PC \leftarrow 2500$



Addr. Mode	Flags Affected	Cycles	T-States
Immediate	None	2/5	9/18

5) RET

(Unconditional Return)

This instruction is written at the end of the sub-routine and enables the control to go back to the main program.

We enter a subroutine using CALL instruction in which we push the return address into the stack.

In RET instruction we do the opposite i.e. we POP the return address from the stack into PC.

Eg: **RET** ; $PC_L \leftarrow [SP]$
 $SP \leftarrow SP + 1$
 $PC_H \leftarrow [SP]$
 $SP \leftarrow SP + 1$

Addr. Mode	Flags Affected	Cycles	T-States
Indirect	None	3	10

6) RCondition

(Conditional Return)

It is the same as UnConditional Return except that the action takes place ONLY if the condition is true.

Eg: RC ; IF C = 1 then
PC_L ← [SP]
SP ← SP + 1
PC_H ← [SP]
SP ← SP + 1

#Please refer Bharat Sir's Lecture Notes for this ...

Addr. Mode	Flags Affected	Cycles	T-States
Indirect	None	1/3	6/12



Bharat
Acharya

7) RSTn

(Restart n)

This instruction is very similar to the CALL instruction.

Here the branch address, instead of being specified directly in the instruction, is calculated as $(n \times 8)$.

The current PC is Pushed into the stack.

The new value of PC is $(n \times 8)$.

The value of $n = 0, 1, 2 \dots 7$.

These instructions are called as Software Interrupts.

Operationally it is thus similar to CALL except that it is a **1 byte instruction**.

Eg: RST1 ; SP ← SP - 1
[SP] ← PC_H
SP ← SP - 1
[SP] ← PC_L
PC ← 0008 ($\because 1 \times 8 = 0008$)

#Please refer Bharat Sir's Lecture Notes for this ...

Addr. Mode	Flags Affected	Cycles	T-States
Indirect	None	3	12

Special Note:

PCHL also causes a branch in the program flow (to the location pointed by the HL Pair), but is already included in the data transfer group. It is a very important instruction and during programming, you should remember that it can also cause a branch.